

EVENTIFY — A CONTAINERIZED MICROSERVICES ARCHITECTURE FOR EVENT TICKETING



AUTHORS

Carlos A. Abella · Daniel F. Páez · Leidy M. Morales

AFFILIATIONS

Systems Engineering — Universidad Distrital Francisco José de Caldas

INTRODUCTION

Eventify addresses the limitations of monolithic ticketing platforms by implementing a containerized microservices architecture. Traditional systems frequently combine authentication, business rules and frontend concerns, producing operational coupling and deployment fragility. This work presents the final architectural solution, emphasizing service separation, reproducible builds and automated CI/CD pipelines that support reliable staging and iterative delivery.

GOAL

Research question: Can a microservices-based architecture improve scalability, testability and deployment reproducibility for an event ticketing platform?

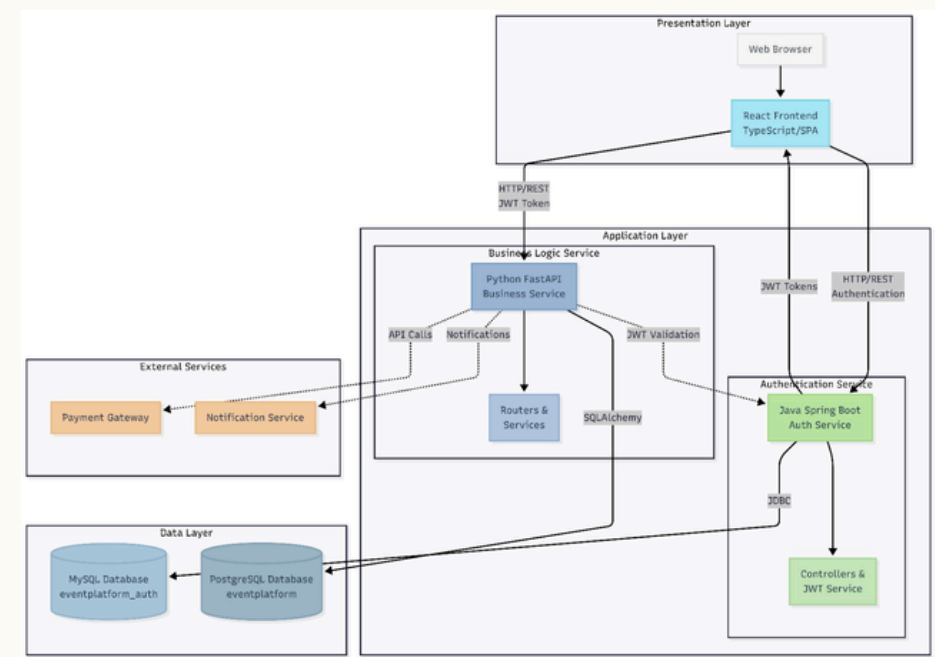
Deliverable: a working, containerized prototype (Eventify) composed of an Authentication service (Java), a Business service (Python) and a React frontend, validated through automated CI and load testing.

PROPOSED SOLUTION

Eventify is organized into three independently deployable components: (1) Authentication Service (Java Spring Boot) for user management and JWT issuance (MySQL), (2) Business Service (Python FastAPI) for events, ticket inventory and order processing (PostgreSQL), and (3) React SPA (Nginx) for the user interface. Services expose clear REST APIs and communicate via stateless JWT tokens. All components are packaged with multi-stage Dockerfiles; GitHub Actions executes tests and builds, producing immutable image artifacts deployed to staging via Docker Compose (or orchestrator).

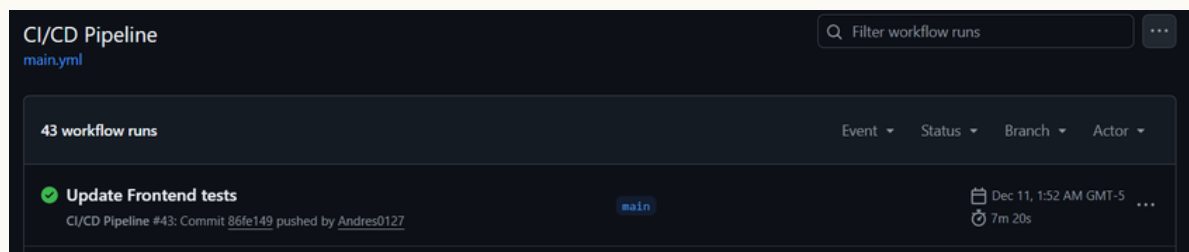
Technical considerations (one-line bullets):

- Dedicated DB per service for data isolation and independent backup policies.
- Healthchecks, restart policies and network isolation at the orchestration layer.
- Multi-stage Docker builds to minimize image size and improve security.
- CI pipeline enforces tests → build → tag → push.



DEPLOYMENT & CI/CD

The CI/CD pipeline (GitHub Actions) runs unit and integration tests for each component, executes database-backed integration scenarios in ephemeral containers, and builds multi-stage production images tagged with commit SHAs. Successful builds are pushed to a container registry and deployed to a staging environment using Docker Compose. This workflow ensures reproducible artifacts, short feedback loops and traceability from commit to image.



EXPERIMENTS

Validation covered functional correctness (unit & integration), acceptance scenarios, and load experiments. Tests were executed inside the CI environment to ensure parity between developer and CI runs.

Validation Activity	Tooling	Key metric / outcome
Unit tests	JUnit / pytest	Coverage ≈ 78–85%
Integration tests	DB containers (MySQL, Postgres)	ORM mappings & transactions validated
Acceptance	Cucumber / manual flows	Authentication and basic purchase scenarios passed
Load tests	Apache JMeter	Avg latency (10/50/100 users): 120 / 280 / 450 ms ; error rate < ~1–2% at peak

RESULTS

- Reproducibility: Docker + CI produced identical images across environments.
- Isolation: Service separation reduced test flakiness and prevented cross-service failures.
- Performance: Stable responses <300 ms under ~50 concurrent users; graceful degradation beyond.
- Deployment: Average CI run: 8–12 minutes; success rate ≈ 95%.

CONCLUSIONS

Eventify demonstrates that a container-first microservices architecture combined with an automated CI/CD pipeline yields measurable improvements in deployability, isolation and development velocity. The prototype satisfies the project goals for reproducible deployment and validated functionality. Future work will integrate a production-grade payment gateway, centralized observability, and cloud autoscaling experiments.

References:

- [1] C. Richardson, Microservices Patterns: With Examples in Java, Manning, 2018.
- [2] Abella, Paez, Morales — Eventify: Technical Report, Universidad Distrital, 2025.
- [3] Abella, Paez, Morales — Eventify: A Microservices-Based Event Management Platform (project paper, 2025).